TOP 50 Nodejs Code Based Interview Questions

Created By - https://www.linkedin.com/in/ankit7rma/

1. Basic Node.js Concepts

Q1: Print "Hello, World!" to the console

Answer:

```
console.log("Hello, World!");
```

Q2: Create a module that exports a function to add two numbers

Answer:

add.js

```
function add(a, b) {
   return a + b;
}
module.exports = add;
```

main.js

```
const add = require('./add');
console.log(add(5, 3)); // Output: 8
```

Q3: Read a file asynchronously and print its content

Answer:

```
const fs = require('fs');

fs.readFile('example.txt', 'utf8', (err, data) ⇒ {
   if (err) {
      console.error(err);
      return;
   }
   console.log(data);
});
```

Q4: Write "Node.js is awesome!" to a file

Answer:

```
const fs = require('fs');

fs.writeFile('output.txt', 'Node.js is awesome!', (err) ⇒ {
   if (err) throw err;
   console.log('File created!');
});
```

Q5: Implement an event emitter with a custom event

Answer:

```
const EventEmitter = require('events');

const myEmitter = new EventEmitter();

myEmitter.on('greet', (name) ⇒ {
   console.log(`Hello, ${name}!`);
});

myEmitter.emit('greet', 'John');
```

2. File System (fs Module)

Q6: Check if a file exists, and create it if it doesn't

Answer:

```
const fs = require('fs');

if (!fs.existsSync('newfile.txt')) {
    fs.writeFileSync('newfile.txt', 'This is a new file');
    console.log('File created!');
} else {
    console.log('File already exists');
}
```

Q7: Delete a file if it exists

Answer:

```
const fs = require('fs');

if (fs.existsSync('file-to-delete.txt')) {
    fs.unlinkSync('file-to-delete.txt');
    console.log('File deleted');
} else {
    console.log('File not found');
}
```

Q8: Append data to a file

Answer:

```
const fs = require('fs');
```

```
fs.appendFile('append.txt', '\nMore data', (err) ⇒ {
  if (err) throw err;
  console.log('Data appended');
});
```

Q9: List all files in a directory

Answer:

```
const fs = require('fs');

fs.readdir('./', (err, files) ⇒ {
  if (err) throw err;
  console.log(files);
});
```

Q10: Rename a file

Answer:

```
const fs = require('fs');

fs.rename('old.txt', 'new.txt', (err) ⇒ {
  if (err) throw err;
  console.log('File renamed!');
});
```

---

3. HTTP Server (Using Core Node.js)

Q11: Create a basic HTTP server

Answer:

```
const http = require('http');

const server = http.createServer((req, res) ⇒ {
   res.writeHead(200, { 'Content-Type': 'text/plain' });
   res.end('Welcome to my server!');
});

server.listen(3000, () ⇒ console.log('Server running on port 3000'));
```

Q12: Handle different routes in a server

Answer:

```
const http = require('http');

const server = http.createServer((req, res) ⇒ {
   if (req.url === '/') {
      res.end('Home Page');
   } else if (req.url === '/about') {
      res.end('About Page');
   } else {
      res.writeHead(404);
      res.end('Page not found');
   }
});

server.listen(3000);
```

---

4. Express.js

Q16: Create an Express server

Answer:

```
const express = require('express');
const app = express();

app.get('/', (req, res) ⇒ {
   res.send('Hello Express!');
});

app.listen(3000, () ⇒ console.log('Server running on port 3000'));
```

Q17: Create Express middleware that logs requests

Answer:

```
const express = require('express');
const app = express();

app.use((req, res, next) ⇒ {
   console.log(`${req.method} request to ${req.url}`);
   next();
});

app.get('/', (req, res) ⇒ res.send('Hello'));

app.listen(3000);
```

5. Asynchronous Programming & Promises

Q21: Fetch data from an API

Answer:

```
const fetch = require('node-fetch');

async function getData() {
```

```
  const response = await fetch('https://jsonplaceholder.typicode.com/todos/
1');
  const data = await response.json();
  console.log(data);
}

getData();
```

Q22: Convert a callback function into a Promise

Answer:

```
const fs = require('fs');

function readFilePromise(filePath) {
  return new Promise((resolve, reject) ⇒ {
    fs.readFile(filePath, 'utf8', (err, data) ⇒ {
      if (err) reject(err);
      else resolve(data);
    });
  });
}

readFilePromise('example.txt').then(console.log).catch(console.error);
```

Q23: Function that waits 2 seconds before printing

Answer:

```
function wait() {
  return new Promise((resolve) ⇒ setTimeout(resolve, 2000));
}

async function delayedMessage() {
  await wait();
```

```
    console.log('Delayed message');
}

delayedMessage();
```

6. Database Operations (MongoDB)

Q31: Connect to MongoDB and retrieve all users

Answer:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/testdb', { useNewUrlParser: tr
ue });

const UserSchema = new mongoose.Schema({ name: String });
const User = mongoose.model('User', UserSchema);

User.find().then(console.log);
```

Q32: Insert a new user

Answer:

```
const newUser = new User({ name: 'Alice' });

newUser.save().then(() ⇒ console.log('User added'));
```

7. API Development

Q36: Implement JWT Authentication

Answer:

```
const jwt = require('jsonwebtoken');

const token = jwt.sign({ userId: 1 }, 'secretKey', { expiresIn: '1h' });
console.log(token);
```

Q40: Upload a file using Multer

Answer:

```
const express = require('express');
const multer = require('multer');
const app = express();
const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('file'), (req, res) ⇒ {
   res.send('File uploaded successfully!');
});

app.listen(3000);
```

8. Error Handling

Q41: Handle errors using try-catch in an async function

Answer:

```
async function fetchData() {
   try {
      let response = await fetch('https://jsonplaceholder.typicode.com/todos/
1');
      let data = await response.json();
      console.log(data);
   } catch (error) {
```

```
        console.error('Error fetching data:', error.message);
    }
}

fetchData();
```

Q42: Handle errors in Express with middleware

Answer:

```
const express = require('express');
const app = express();

app.get('/', (req, res) ⇒ {
    throw new Error('Something went wrong');
});

// Error handling middleware
app.use((err, req, res, next) ⇒ {
    console.error(err.message);
    res.status(500).send('Internal Server Error');
});

app.listen(3000, () ⇒ console.log('Server running on port 3000'));
```

9. WebSockets

Q43: Create a WebSocket server using ws

Answer:

```
const WebSocket = require('ws');

const server = new WebSocket.Server({ port: 8080 });
```

```
server.on('connection', (ws) ⇒ {
    ws.send('Welcome to the WebSocket server!');

    ws.on('message', (message) ⇒ {
        console.log(`Received: ${message}`);
        ws.send(`You said: ${message}`);
    });
});

console.log('WebSocket server running on ws://localhost:8080');
```

Q44: Implement a WebSocket client

Answer:

```
const WebSocket = require('ws');

const client = new WebSocket('ws://localhost:8080');

client.on('open', () ⇒ {
    console.log('Connected to WebSocket server');
    client.send('Hello, server!');
});

client.on('message', (data) ⇒ {
    console.log(`Server says: ${data}`);
});
```

---

10. Caching & Performance Optimization

Q45: Implement caching in Express using Redis

Answer:

```
const express = require('express');
const redis = require('redis');
const app = express();

const client = redis.createClient();

app.get('/data', async (req, res) ⇒ {
   client.get('cachedData', async (err, cachedData) ⇒ {
      if (cachedData) {
         return res.json({ source: 'cache', data: JSON.parse(cachedData) });
      }

      let data = { message: 'This is fresh data' };
      client.setex('cachedData', 60, JSON.stringify(data));
      res.json({ source: 'API', data });
   });
});

app.listen(3000, () ⇒ console.log('Server running on port 3000'));
```

Q46: Use compression middleware in Express

Answer:

```
const express = require('express');
const compression = require('compression');
const app = express();

app.use(compression());

app.get('/', (req, res) ⇒ {
   res.send('This response is compressed!');
});
```

```
app.listen(3000);
```

---

11. Security

Q47: Prevent SQL injection using parameterized queries

Answer:

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
   host: 'localhost',
   user: 'root',
   password: '',
   database: 'testdb'
});

const userId = 1;
connection.query('SELECT * FROM users WHERE id = ?', [userId], (err, result
s) => {
   if (err) throw err;
   console.log(results);
});
```

Q48: Secure Express headers using Helmet

Answer:

```
const express = require('express');
const helmet = require('helmet');
const app = express();

app.use(helmet());
```

```
app.get('/', (req, res) ⇒ res.send('Secure app!'));

app.listen(3000, () ⇒ console.log('Server running on port 3000'));
```

---

12. Testing

Q49: Write a unit test for a function using Jest

Answer:

sum.js

```
function sum(a, b) {
  return a + b;
}

module.exports = sum;
```

sum.test.js

```
const sum = require('./sum');

test('adds 2 + 3 to equal 5', () ⇒ {
  expect(sum(2, 3)).toBe(5);
});
```

Run the test:

```
npm install --save-dev jest
npx jest
```

Q50: Test an Express API route using Supertest

Answer:

app.js

```
const express = require('express');
const app = express();

app.get('/hello', (req, res) ⇒ {
  res.status(200).json({ message: 'Hello, World!' });
});

module.exports = app;
```

app.test.js

```
const request = require('supertest');
const app = require('./app');

test('GET /hello should return Hello, World!', async () ⇒ {
  const response = await request(app).get('/hello');
  expect(response.status).toBe(200);
  expect(response.body.message).toBe('Hello, World!');
});
```

Run the test:

```
npm install --save-dev supertest jest
npx jest
```